



Figure 5: Database snapshots

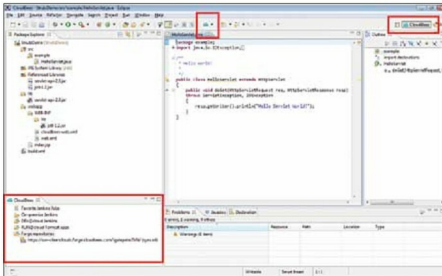


Figure 6: CloudBees Eclipse plug-in/perspective

Next, select **Run as > CloudBees Application (Run @ Cloud)** from the project context menu. Select the account. Access <http://timenexpense.cleanclouds.cloudbees.net/hello>. Commit the revision (an SVN commit will be done). Once the code is committed, you can compare using Java Source Compare functionality.

Application configuration and operations

Users can enable application-specific New Relic RPM application monitoring, Papertrail application logging and AppDynamics monitoring. New Relic RPM provides monitoring, troubleshooting, root cause diagnosis, capacity planning, scalability analysis, database usage analysis, Apex scoring, and deployment history. Papertrail provides consolidated logging for your applications to help detect, resolve, and avoid infrastructure problems. AppDynamics is the leading provider of application performance management for modern application architectures in the cloud.

All CloudBees Run@Cloud applications have an **Operations** page in the **Console** (Web-based) that provides insight into the health and activity of the application. You can get to these charts by logging into the **Console**, selecting one of your applications, and then clicking the **Operations**

tab. There, memory graphs (memory, classes and thread statistics), request graphs (requests, errors, request time and request byte statistics) and overview graphs (requests, memory, sessions and system load) are available.

Server and access logs are available in case they are needed for troubleshooting.

Scaling

In normal scenarios, the user experience suffers due to a lack of resources, or because running too many

servers will cost a lot of money.

It is all about perfect capacity planning—but how do you get that perfection in today's dynamic IT environment? That is where auto-scale, one of the most important features of PaaS, comes to your rescue—it increases and decreases server capacity based on real-time demand. It requires the platform provider to load-balance requests across a number of servers, monitor the load on each server, and spin up new resources as needed. It tries to keep application resources at a 'sweet spot' that saves money, yet ensures the user experience doesn't suffer.

All PaaS providers support auto-scaling to some extent. Java EE applications must be configured to access a centralised external database, as opposed to a database server co-hosted on the same server. With CloudBees, scaling is easy; it is available as a paid subscription. Once enabled, your application management screen will show new options for scaling (Figure 8).

Vertical scaling for the application is achieved through app-cells. Choose the appropriate size. One app-cell offers dedicated RAM and compute processing power for your application to use. The more the app-cells, the more memory and processing power will be available to improve the performance and scalability of the application. For horizontal scaling, the drop-down lets you choose the number of machines to add to scale horizontally. Applications can choose a specific number of instances (up to 10) and set this up statically.

The auto-scale service knows about your application's preferences, which are stored as a list of scale rules. It then listens to the *app.stat* feed (see the diagram in Figure 9) and does stream processing over it. Once it detects that your application has exceeded the set parameters, it will send a message to the deployment API service to add more capacity or reduce it (as required), within the range that you specified (settings in Figure 10). Thus, users need not worry about variations